

Arbitrage Filtering of Option Prices: A Simple Real-Time Approach

Karim Moussa* 

Vrije Universiteit Amsterdam and Tinbergen Institute, the Netherlands

November 3, 2025

Abstract

This paper presents a simple arbitrage filtering method for preprocessing European option quotes. Prices are modified only when basic no-arbitrage conditions are violated, and no quotes are discarded. Instead, quotes are adjusted using information from other, more liquid quotes. This process is iterative and continues until all violations are resolved. The filter is shown to minimize an intuitive objective function, and local optimality properties are established, relating to the most liquid quotes. The method is straightforward to implement and fast, making it particularly suitable for real-time applications. Numerical examples illustrate key filtering properties, including the tendency to produce sparse solutions that leave many quotes unchanged, as well as its robustness to outliers.

Keywords: Arbitrage-free, Call price surface, Local volatility, Model-free bounds, Preprocessing, Volatility surface.

*E-mail: k.moussa@vu.nl. This paper has been accepted for publication in *Quantitative Finance*. The final version will be available at <https://doi.org/10.1080/14697688.2025.2574907>.

1 Introduction

Option price data play a crucial role in numerous applications. They are used to calibrate derivative valuation models, to estimate the market risk of investment banks, and for computing their daily profit-and-loss in line with mark-to-market accounting practice. Representing an aggregation of beliefs from investors, option prices are also widely used for their informational content. Important examples are estimation of the risk-neutral density of the underlying asset (Malz, 1997), measuring the degree of risk aversion of market participants (Bliss & Panigirtzoglou, 2004), and their use as explanatory variables in time series models (Koopman, Jungbacker, & Hol, 2005).

Since the rising popularity in the 1970s, the field of option pricing has focused primarily on relative valuation. This approach tries to match the available market prices as closely as possible and, taking these prices as a given, uses their information to determine the value of other derivatives (e.g., Schoutens 2003; Gatheral 2011). The focus on relative pricing offers an explanation of why data quality has historically received little attention. In recent years, however, the preprocessing of option prices has been gaining attention in the literature. For instance, Bams, Blanchard, and Lehnert (2015, p.1) emphasize the importance of preprocessing to valuation, noting that “*the choice of the underlying calibration sample is [...] a crucial step in an option pricing exercise.*”

Bams et al. (2015) also note that the literature is characterized by many different approaches to preprocessing, with most studies focusing on excluding option prices based on their contract parameters, such as the strike price and the expiry date of the option. Examples of such exclusion filters are found in Brandt and Wu (2002) and Barone-Adesi, Engle, and Mancini (2008), who use only out-of-the-money options, Bollen and Whaley (2004), who discard options having an absolute value of Black-Scholes delta outside of a specified range, and Ivanovas and Meier (2015), who omit option prices outside a certain strike range that is determined based on market activity. A different approach is taken by Haven, Liu, and Shen (2012), who apply smoothing by the wavelet method to de-noise option prices. A shared characteristic of the above approaches is that they always make adjustments to the original data set—irrespective of its quality.

Across scientific disciplines, it is generally accepted that preprocessing methods should preserve as much of the original data and inherent information as possible. Consistent with this principle, a more conservative approach to preprocessing option prices is needed. For instance, Bakshi, Cao, and Chen (1997) and Neumann and Skiadopoulos (2013), besides using several exclusion rules based on contract parameters, choose to omit option prices that violate certain arbitrage conditions. For example, an option price below its intrinsic value signals an error in the data. A related but more cautious approach is adopted by Jäckel (2014), who also excludes options, but solely based on arbitrage restrictions. Arguably, the methods most in line with the directive outlined above are those of Le Floc’h and Oosterlee (2019a, 2019b) and Cohen, Reisinger, and Wang (2020), who frame the preprocessing task as an optimization problem designed to minimize the deviation from the original data under arbitrage constraints. Importantly, these methods do not discard any data.

This paper presents a simple arbitrage filtering method for preprocessing European option quotes, which aligns with the above directive and recent literature. Specifically, prices are modified only when basic no-arbitrage conditions are violated, and no quotes are

discarded. Instead, quotes are adjusted using information from other, more liquid quotes. This process is iterative and continues until all violations are resolved. Theoretically, the filter is shown to minimize an intuitive objective function, and local optimality properties are established, relating to the most liquid quotes. From a practical viewpoint, the method is straightforward to implement and fast, making it particularly suitable for real-time applications. Numerical examples illustrate key filtering properties, including the tendency to produce sparse solutions that leave many quotes unchanged, as well as its robustness to outliers. Beyond its general use as a preprocessing technique, the filter can enhance standard interpolation methods and offer a solution for other approaches requiring arbitrage-free quotes as input (e.g., Kahalé 2004; Wang, Yin, and Qi 2004; Malz 2014; Le Floc’h and Oosterlee 2019a, 2019b).

The remainder of this paper is structured as follows. Section 2 discusses several no-arbitrage conditions for European options. Section 3 introduces the filtering algorithm. Section 4 establishes several theoretical properties of the filter. Section 5 presents an extension of the filtering algorithm to account for arbitrage restrictions in the expiry dimension. Section 6 provides numerical examples using real data sets, to illustrate and discuss several key aspects of the method. Section 7 provides discussion and concludes the paper. The appendix contains proofs and derivations.

2 No-arbitrage conditions

This section describes several conditions that hold for any arbitrage-free European call price function, many of which were established in the seminal work of Merton (1973). We denote the call price function by $C(K, T)$, where K is the strike price and T the expiry date in calendar years. As a simple first example, denoting by S_T the value of the underlying asset at time T , the payoff function

$$(S_T - K)^+ = \max\{S_T - K, 0\}$$

implies that European call prices must be decreasing in the strike.

In the following, we assume frictionless markets, as well as the availability of risk-free zero-coupon bonds and forward contracts corresponding to any expiry date $T \in \mathbb{R}_+$. The bond delivers one unit of (domestic) currency for a price of $B(T)$, while the forward delivers one unit of the underlying asset in exchange for $F(T)$ units of currency at the agreed-upon delivery date.

2.1 Conditions in the strike dimension

The property of monotonicity in the strike price can be made more precise by noting that a European digital option, which has payoff

$$\mathbb{I}(S_T \geq K) = \begin{cases} 1 & \text{if } S_T \geq K, \\ 0 & \text{otherwise,} \end{cases}$$

can be replicated using a call spread, by letting the difference between strikes go to zero. This implies

$$0 \leq -\frac{\partial C}{\partial K} = \lim_{h \rightarrow 0} \frac{C(K) - C(K + h)}{h} \leq B(T),$$

with $C(K) = C(K, T)$ for conciseness, and where the bond price $B(T)$ forms an upper bound on the price of any digital option, which follows from its payoff function. We thus obtain the following bounds on the derivative with respect to the strike:

$$-B(T) \leq \frac{\partial C}{\partial K} \leq 0. \quad (1)$$

In addition, call prices must be convex in the strike,

$$\frac{\partial^2 C}{\partial K^2} = \lim_{h \rightarrow 0} \frac{C(K+h) - 2C(K) + C(K-h)}{h^2} \geq 0, \quad (2)$$

since the numerator is a butterfly spread, which has a positive payoff. The call option with strike $K = 0$ has price given by

$$C(0, T) = B(T)F(T), \quad (3)$$

because it is equivalent to a forward contract for which the buyer has to pay at the start of the contract. Combining (1), (2), and (3) with non-negativity of the payoff gives the following European call price bounds:

$$B(T)(F(T) - K)^+ \leq C(K, T) \leq B(T)F(T). \quad (4)$$

Where the non-negativity restriction applies to a single contract, the monotonicity and convexity conditions together ensure the absence of arbitrage opportunities between call options along the strike dimension (e.g., Kahalé, 2004, Lemma 1). Additionally, the lower bound on the derivative in (1), the zero-strike call price in (3), and the bounds in (4) guarantee consistency with the prices of the risk-free bond and the forward contract.

2.2 Conditions in the expiry dimension

In the expiry dimension, a well-known monotonicity condition for European call prices applies when the level of moneyness,

$$\mathcal{K} = K/F(T),$$

is held fixed. Defining the *normalized call price* as a function of moneyness by

$$\mathcal{C}(\mathcal{K}, T) = \frac{C(\mathcal{K} \cdot F(T), T)}{C(0, T)} = \frac{C(K, T)}{C(0, T)} \in [0, 1], \quad (5)$$

it can be shown (e.g., Gatheral 2004; Fengler 2009) that, for two expiries $T_1 < T_2$,

$$\mathcal{C}(\mathcal{K}, T_1) \leq \mathcal{C}(\mathcal{K}, T_2) \quad (6)$$

is necessary and sufficient to exclude arbitrage from calendar spreads, which involve a long position in $\mathcal{C}(\mathcal{K}, T_2)$ and a short position in $\mathcal{C}(\mathcal{K}, T_1)$.

In real data sets, option quotes are typically not arranged on a rectangular grid of moneyness and expiry values, so that direct verification of the above condition for a given set of option prices can be challenging in practice. However, it may be combined with the strike-dimension conditions from the previous section to yield more easily verifiable restrictions. We consider the following result, which integrates naturally into the proposed arbitrage filtering method.

Proposition 1. *Let $T_1 < T_2$ be two expiries with option quotes satisfying the no-arbitrage conditions in (1)–(4). Denote by $\underline{\mathcal{C}}(\mathcal{K}, T_1)$ and $\bar{\mathcal{C}}(\mathcal{K}, T_2)$ the lower and upper bounds implied by these conditions at T_1 and T_2 , respectively, as detailed in Section 3.2. Then,*

$$\underline{\mathcal{C}}(\mathcal{K}, T_1) \leq \bar{\mathcal{C}}(\mathcal{K}, T_2) \quad (7)$$

is necessary for the existence of an arbitrage-free call price function $\mathcal{C}(\mathcal{K}, T)$, $\mathcal{K}, T \in \mathbb{R}_+$, consistent with these quotes.

This result shows that if the inequality in (7) is violated for any moneyness value $\mathcal{K}$, arbitrage-free interpolation of the quotes is not possible. It is therefore particularly relevant in applications that require a continuous surface of prices derived from the given quotes, such as market making in vanilla options or constructing the local volatility surface for the valuation of path-dependent derivatives.

The above result facilitates comparing option quotes between expiries. To this end, suppose we are considering whether a quote with price $\mathcal{C}(\mathcal{K}, T)$ for moneyness $\mathcal{K}$ and expiry T is feasible relative to other quotes. Under frictionless markets (i.e., no bid-ask spreads), accepting this quote would collapse its feasible range to a single point:

$$\underline{\mathcal{C}}(\mathcal{K}, T) = \mathcal{C}(\mathcal{K}, T) = \bar{\mathcal{C}}(\mathcal{K}, T), \quad (8)$$

where $\underline{\mathcal{C}}(\mathcal{K}, T)$ and $\bar{\mathcal{C}}(\mathcal{K}, T)$ represent the bounds for expiry T at moneyness $\mathcal{K}$ implied by (1)–(4). Therefore, Proposition 1 implies that we can directly compare the quote $\mathcal{C}(\mathcal{K}, T)$ with, for example, the lower bound $\underline{\mathcal{C}}(\mathcal{K}, T_1)$ at expiry T_1 when $T_1 < T$, which follows from setting $T_2 = T$ in (7). This illustrates how the result is applied in the extended arbitrage filter presented in Section 5.

The inequality in Proposition 1 is a necessary condition. In the special case where the quotes form a rectangular grid of moneyness and expiry values, it coincides with the condition in (6), which follows from the reasoning leading to (8). In such cases, the set of conditions in (1)–(4) and (7) is also sufficient for the existence of an arbitrage-free call price function consistent with the quotes (e.g., Davis & Hobson, 2007, Corollary 4.1). However, in general, additional conditions may need to be satisfied. A more complete treatment of sufficient conditions for the absence of arbitrage on the call price surface can be found in Cousot (2007, Sec.4) and Cohen et al. (2020, Sec.2.3).

Table 1 collects the conditions that are considered during preprocessing, expressed in terms of the normalized call price as a function of moneyness and expiry. As with the call price as a function of strike, the normalized call price $\mathcal{C}(\mathcal{K}, T)$ is decreasing and convex

Table 1: No-arbitrage constraints on the normalized call price $\mathcal{C}(\mathcal{K}, T)$ defined in (5) as a function of moneyness $\mathcal{K} = K/F(T)$ and expiry T (or T_1 and T_2).

Description	Constraint
Monotonicity ($\mathcal{K}$)	$-1 \leq \partial \mathcal{C} / \partial \mathcal{K} \leq 0$
Convexity ($\mathcal{K}$)	$\partial^2 \mathcal{C} / \partial \mathcal{K}^2 \geq 0$
Price bounds	$(1 - \mathcal{K})^+ \leq \mathcal{C}(\mathcal{K}, T) \leq 1$
Monotonicity ($T_1 < T_2$)	$\underline{\mathcal{C}}(\mathcal{K}, T_1) \leq \bar{\mathcal{C}}(\mathcal{K}, T_2)$

in the moneyness, $\mathcal{K}$. In particular, the bounds on the derivative follow from

$$\frac{\partial \mathcal{C}}{\partial \mathcal{K}} = \frac{1}{C(0, T)} \cdot \frac{\partial C}{\partial \mathcal{K}} = \frac{1}{B(T)F(T)} \cdot \frac{\partial C}{\partial K} \cdot \frac{\partial K}{\partial \mathcal{K}}, \quad \text{where} \quad \frac{\partial K}{\partial \mathcal{K}} = \left(\frac{\partial \mathcal{K}}{\partial K} \right)^{-1} = F(T).$$

3 Arbitrage filtering of option prices

3.1 Filtering in the strike dimension

This section describes a filtering procedure for removing arbitrage along the strike (or moneyness) dimension from a set of quotes with the same expiry. We assume the availability of N quotes

$$Q_i = (\mathcal{K}_i, \mathcal{C}_i), \quad \mathcal{C}_i = \mathcal{C}(\mathcal{K}_i) = \mathcal{C}(\mathcal{K}_i, T), \quad i = 1, \dots, N,$$

where the expiry argument T will at times be omitted for conciseness. The idea is to first sort the given quotes based on their liquidity and, starting with the most liquid quote onwards, consecutively check each one for feasibility relative to a set of arbitrage-free quotes, $\mathbb{A}$. Each feasible quote is directly added to $\mathbb{A}$ before the next check, so that the implied feasible range becomes tighter as the arbitrage-free set grows, reflecting the additional information from the accepted quotes. The order in which the quotes are processed embodies the notion that their accuracy tends to improve with liquidity. After all quotes have been considered, those initially excluded are adjusted and added to $\mathbb{A}$, which contains quotes for all original moneyness values $\mathcal{K}_i$, $i = 1, \dots, N$.

Algorithm 1 presents the arbitrage filtering method. The filter is initialized by filling the arbitrage-free set with the zero-strike quote, which is implied by theory. In the second step, the quotes are sorted based on a chosen proxy for liquidity, which can be selected according to preference. Common examples include the inverse bid-ask spread, the traded volume, or the open interest. A detailed discussion of alternatives can be found in Le and Gregoriou (2020) and Foucault, Pagano, and Röell (2023, Ch. 2).

In practice, it may be that no liquidity information is available. Important examples are when the option prices originate from the over-the-counter market, in which trading is not centralized, and when using synthetic price data, as is standard practice for estimating the risk of a derivative portfolio (Embrechts, Frey, & McNeil, 2015). In these cases, a common solution is to use the moneyness of an option as a proxy for liquidity, based on the observation that liquidity tends to decrease as the strike becomes more distant from the forward. To illustrate the arbitrage filtering method, we apply a simple rule that ranks the quotes based on the liquidity proxy

$$\ell_i = \frac{1}{|K_i - F(T)| + 1} \in (0, 1],$$

which attains its maximum value of one when the strike equals the forward. The first-ranked quote $Q_{(1)}$ has the highest ℓ_i value—usually the at-the-money (ATM) quote—and the last-ranked quote $Q_{(N)}$ corresponds to the lowest liquidity proxy.

In Step 3, each quote $Q_{(i)}$ is consecutively checked for a violation of the bounds $\underline{\mathcal{C}}$ and $\bar{\mathcal{C}}$ implied by $\mathbb{A}$ and directly added if feasible. To maintain the flow of the discussion, the bounds are derived in the next section. Step 4 iteratively modifies the quotes that

Algorithm 1 Arbitrage filtering method for option quotes with same expiry date

1. **Initialize:** Define the set of arbitrage-free quotes as

$$\mathbb{A} = \{Q_0\},$$

with $Q_0 = (0, 1)$ the quote corresponding to a strike of zero (implied by theory).

2. **Sort:** Rank the given quotes based on a chosen proxy for liquidity, resulting in a sorting

$$Q_{(1)}, \dots, Q_{(N)},$$

with $Q_{(1)} = (\mathcal{K}_{(1)}, \mathcal{C}_{(1)})$ corresponding to the highest value for the liquidity proxy.

3. **Check:** For $i = 1, \dots, N$: add $Q_{(i)}$ to $\mathbb{A}$ if

$$\underline{\mathcal{C}}(\mathcal{K}_{(i)}) \leq \mathcal{C}_{(i)} \leq \bar{\mathcal{C}}(\mathcal{K}_{(i)}), \quad (9)$$

with $\underline{\mathcal{C}}(\mathcal{K})$ and $\bar{\mathcal{C}}(\mathcal{K})$ the lower and upper bounds implied by the quotes in $\mathbb{A}$, respectively.

4. **Adjust:** For $i = 1, \dots, N$: if $Q_{(i)} \notin \mathbb{A}$, define the adjusted value

$$\mathcal{C}_{(i)}^* = \begin{cases} \underline{\mathcal{C}} + \lambda (\bar{\mathcal{C}} - \underline{\mathcal{C}}) & \text{if } \mathcal{C}_{(i)} < \underline{\mathcal{C}}, \\ \underline{\mathcal{C}} + (1 - \lambda) (\bar{\mathcal{C}} - \underline{\mathcal{C}}) & \text{otherwise,} \end{cases} \quad (10)$$

for some $\lambda \in [0, 1]$, with $\underline{\mathcal{C}} = \underline{\mathcal{C}}(\mathcal{K}_{(i)})$ and $\bar{\mathcal{C}} = \bar{\mathcal{C}}(\mathcal{K}_{(i)})$. Add $Q_{(i)}^* = (\mathcal{K}_{(i)}, \mathcal{C}_{(i)}^*)$ to $\mathbb{A}$.

5. **Terminate:** Return the quotes in $\mathbb{A}$.
-

were originally excluded from $\mathbb{A}$ by choosing an adjusted value inside the feasible range. Expression (10) reformulates this problem in the intuitive form of choosing a convex combination of the bounds with weight parameter $\lambda \in [0, 1]$. Here, $\lambda = 0$ coincides with the bound that was violated, and $\lambda = 1$ corresponds to the opposite bound. Setting $\lambda = 0$ minimizes the immediate impact and is generally adequate; as illustrated below, the feasible price range is typically quite tight after Step 3, which limits the sensitivity to this parameter. Further discussion on the choice of λ is provided in Section 4.2.

Figure 1 illustrates the filtering procedure step-by-step for the data from Table 2, with $T = 1$ and $F(T) = B(T) = 1$, so that $\mathcal{K} = K$ and $\mathcal{C}(\mathcal{K}, T) = C(K, T)$. The top-left plot shows the original data alongside the European call price bounds in (4). The data set contains arbitrage opportunities, as the call prices are neither convex nor decreasing in strike. The next plot shows the arbitrage-free set after the first-ranked quote has been added, $\mathbb{A} = \{Q_0, Q_{(1)}\}$, as well as the implied bounds, which are drastically tightened. The top-right plot shows that the second-ranked quote is within the feasible call price range, hence it is added to $\mathbb{A}$, as shown in the leftmost plot in the middle row. The third-ranked quote is also feasible and added to $\mathbb{A}$. The next two plots on the middle row show that the fourth and fifth-ranked quotes are initially rejected because they fall outside of the feasible call price range implied by the quotes in $\mathbb{A}$. The bottom row shows how these quotes are adjusted to the implied bounds and consecutively added to $\mathbb{A}$.

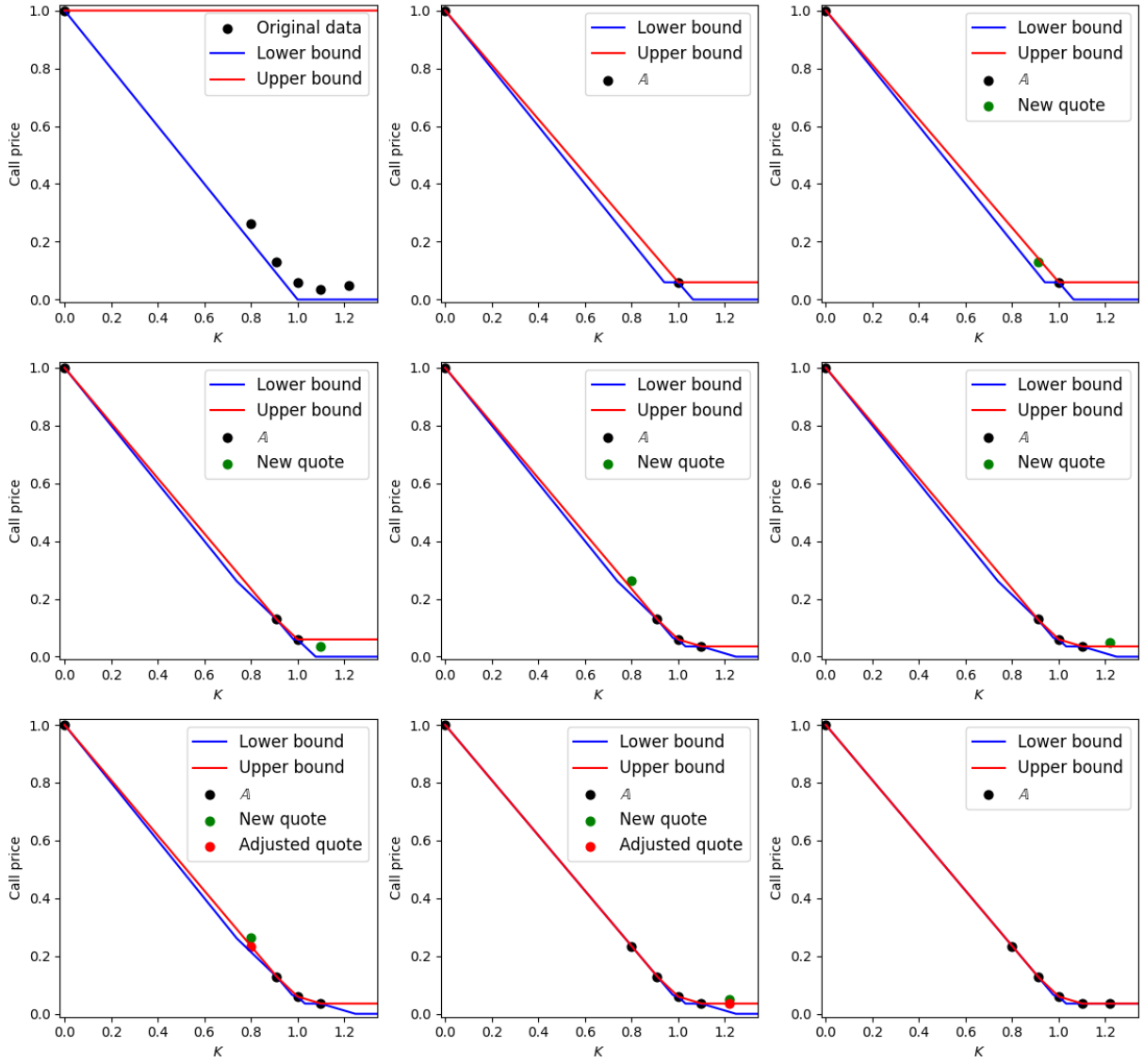


Figure 1: Arbitrage filtering method illustrated step-by-step for the data from Table 2, with $\lambda = 0$. The original call prices are neither convex (the three leftmost quotes), nor decreasing (the two rightmost quotes) in the strike. Both violations are resolved in the filtered data set.

Table 2: Example price data with $T = 1$ and $F(T) = B(T) = 1$, and corresponding filtered prices based on Algorithm 1 with $\lambda = 0$. In this case, $\mathcal{K} = K$ and $\mathcal{C}(\mathcal{K}, T) = C(K, T)$.

Moneyness	0.80	0.91	1.0	1.10	1.22
Liquidity rank (i)	4	2	1	3	5
Call price (original)	0.26	0.13	0.06	0.04	0.05
Call price (filtered)	0.23	0.13	0.06	0.04	0.04

3.2 Derivation of the bounds

The bounds $\underline{\mathcal{C}}(\mathcal{K})$ and $\overline{\mathcal{C}}(\mathcal{K})$ represent the feasible range of the normalized call price implied by the quotes in $\mathbb{A}$ via the conditions in (1)–(4), as expressed in Table 1. While checking a quote for feasibility with respect to the arbitrage-free set, we distinguish between whether or not it is enclosed by two points from $\mathbb{A}$ in terms of moneyness. We first consider the case in which it is not, which applies, for example, to the first-ranked quote, $Q_{(1)}$. For such a quote with moneyness $\mathcal{K}$ and price $\mathcal{C}(\mathcal{K})$, the conditions of monotonicity and convexity relative to $\mathbb{A}$ are satisfied if

$$\underline{D}(\mathcal{K}) \leq \frac{\mathcal{C}(\mathcal{K}) - \mathcal{C}(\mathcal{K}_L)}{\mathcal{K} - \mathcal{K}_L} \leq 0,$$

where $\mathcal{K}_L$ denotes the nearest moneyness to the left of $\mathcal{K}$ corresponding to a quote in $\mathbb{A}$, $\mathcal{C}(\mathcal{K}_L)$ is the corresponding normalized call price, and $\underline{D}(\mathcal{K})$ is defined to equal -1 if $\mathcal{K}_L = 0$, or as the difference quotient of the two nearest quotes in $\mathbb{A}$ to the left of $\mathcal{K}$ otherwise. The right-hand inequality implies the upper bound

$$\overline{\mathcal{C}}(\mathcal{K}) = \mathcal{C}(\mathcal{K}_L),$$

while the left-hand inequality, combined with non-negativity, implies the lower bound

$$\underline{\mathcal{C}}(\mathcal{K}) = \left(\mathcal{C}(\mathcal{K}_L) + \underline{D}(\mathcal{K}) \cdot (\mathcal{K} - \mathcal{K}_L) \right)^+.$$

For the other case, let $\mathcal{K}_R$ denote the nearest moneyness to the right of $\mathcal{K}$ for a quote in $\mathbb{A}$, with $\mathcal{C}(\mathcal{K}_R)$ the corresponding price. In addition, define $\overline{D}(\mathcal{K})$ to equal zero if there is precisely one quote in $\mathbb{A}$ to the right of $\mathcal{K}$; if there are more such quotes, $\overline{D}(\mathcal{K})$ is the difference quotient of the two nearest quotes to the right. The conditions of monotonicity and convexity relative to $\mathbb{A}$ are satisfied if

$$\underline{D}(\mathcal{K}) \leq \frac{\mathcal{C}(\mathcal{K}) - \mathcal{C}(\mathcal{K}_L)}{\mathcal{K} - \mathcal{K}_L} \leq \frac{\mathcal{C}(\mathcal{K}_R) - \mathcal{C}(\mathcal{K})}{\mathcal{K}_R - \mathcal{K}} \leq \overline{D}(\mathcal{K}). \quad (11)$$

The left and rightmost inequalities in (11) imply lower bounds for $\mathcal{C}(\mathcal{K})$ given by

$$\underline{\mathcal{C}}_{\underline{D}}(\mathcal{K}) = \left(\mathcal{C}(\mathcal{K}_L) + \underline{D}(\mathcal{K}) \cdot (\mathcal{K} - \mathcal{K}_L) \right)^+,$$

and

$$\underline{\mathcal{C}}_{\overline{D}}(\mathcal{K}) = \mathcal{C}(\mathcal{K}_R) - \overline{D}(\mathcal{K}) \cdot (\mathcal{K}_R - \mathcal{K}).$$

The lower bounds $\underline{\mathcal{C}}_{\underline{D}}(\mathcal{K})$ and $\underline{\mathcal{C}}_{\overline{D}}(\mathcal{K})$ are the prices attained with linear extrapolation from $(\mathcal{K}_L, \mathcal{C}(\mathcal{K}_L))$ and $(\mathcal{K}_R, \mathcal{C}(\mathcal{K}_R))$ using difference quotients $\underline{D}(\mathcal{K})$ and $\overline{D}(\mathcal{K})$ as slopes, respectively. The lower bound $\underline{\mathcal{C}}(\mathcal{K})$ is given by the maximum of these two lower bounds:

$$\underline{\mathcal{C}}(\mathcal{K}) = \max \{ \underline{\mathcal{C}}_{\underline{D}}(\mathcal{K}), \underline{\mathcal{C}}_{\overline{D}}(\mathcal{K}) \}.$$

The upper bound is implied by the middle inequality in (11), which is the point attained using linear interpolation between $(\mathcal{K}_L, \mathcal{C}(\mathcal{K}_L))$ and $(\mathcal{K}_R, \mathcal{C}(\mathcal{K}_R))$:

$$\overline{\mathcal{C}}(\mathcal{K}) = \frac{(\mathcal{K}_R - \mathcal{K})\mathcal{C}(\mathcal{K}_L) + (\mathcal{K} - \mathcal{K}_L)\mathcal{C}(\mathcal{K}_R)}{\mathcal{K}_R - \mathcal{K}_L}.$$

3.3 Incorporating bid and ask prices

When bid and ask prices are available for the options, they can straightforwardly be incorporated into Algorithm 1 as follows. Denote the normalized bid and ask call prices for moneyness $\mathcal{K}$ by $\mathcal{C}_{\text{Bid}}(\mathcal{K})$ and $\mathcal{C}_{\text{Ask}}(\mathcal{K})$. In Step 3, when checking if the (i) -th quote satisfies the arbitrage restrictions implied by $\mathbb{A}$, we first consider whether our reference price, which is chosen here for illustration as the mid-price

$$\mathcal{C}_{\text{Mid}}(\mathcal{K}_{(i)}) = \frac{\mathcal{C}_{\text{Bid}}(\mathcal{K}_{(i)}) + \mathcal{C}_{\text{Ask}}(\mathcal{K}_{(i)})}{2},$$

satisfies the bounds in (9). If it does, the mid-quote is added to $\mathbb{A}$. If not, we check whether there are elements in the intersection

$$[\mathcal{C}_{\text{Bid}}(\mathcal{K}_{(i)}), \mathcal{C}_{\text{Ask}}(\mathcal{K}_{(i)})] \cap [\underline{\mathcal{C}}, \overline{\mathcal{C}}].$$

If this interval is non-empty, we select the element nearest to the reference price and add it to $\mathbb{A}$. Otherwise, we continue to the next iteration. The remaining steps of the algorithm proceed as usual.

The above approach has the effect that more liquid quotes are rejected less often because it is now possible to accept any point in the bid-ask range, instead of just a single point. When the reference price is rejected, accepting a different price within the bid-ask range in Step 3 will generally lead to a smaller adjustment than the correction applied in Step 4 via (10), where the adjusted price is determined based on the feasible range implied by other quotes. A related approach is considered by Cohen et al. (2020, Sec. 3.2), who distinguish between soft and hard constraints in their optimization procedure, depending on whether adjustments are inside or outside the bid-ask range.

3.4 Computational considerations

3.4.1 Computational complexity

Table 3 presents the computational complexity of Algorithm 1 applied to a data set with N option quotes for a given expiry. Step 1 is a constant-time operation. Step 2 is $O(N \log(N))$, which reflects the complexity of asymptotically optimal comparison sorts, attained by well-known procedures such as merge sort and heapsort (Cormen, Leiserson, Rivest, & Stein, 2022, p.193). Steps 3 and 4 involve the possible insertion and search for adjacent quotes in the arbitrage-free set $\mathbb{A}$ for the option quotes considered. These

Table 3: Computational complexity of Algorithm 1 applied to N option quotes. Step 1 is a constant-time operation. Step 2 uses an asymptotically optimal comparison sort (e.g., merge sort or heapsort). Steps 3 and 4 rely on a balanced search tree to store and operate on the arbitrage-free set $\mathbb{A}$. Step 5 assumes that the processed quotes are returned as an array, corresponding to an inorder tree walk.

Step	Step 1	Steps 2–4	Step 5	Overall
Complexity	$O(1)$	$O(N \log(N))$	$O(N)$	$O(N \log(N))$

operations can be performed in $O(N \log(N))$ time by relying on a balanced search tree (e.g., Cormen et al., 2022, Ch.13) to efficiently store and operate on $\mathbb{A}$.

In particular, both insertion and finding the predecessor or successor are $O(M)$ operations for a tree of height M . Using a balanced tree ensures that $M \propto \log(N)$ with N elements; performing the above operations for N quotes therefore yields an $O(N \log(N))$ complexity. This improves over standard array structures, which incur quadratic complexity in N , as sorted insertion requires shifting subsequent elements.

Lastly, Step 5 assumes that the processed quotes are returned as an array. This corresponds to an inorder tree walk, which is $O(N)$ for a search tree with N elements (Cormen et al., 2022, Theorem 12.1). The overall complexity of Algorithm 1 is therefore

$$O(1) + 3O(N \log(N)) + O(N) = O(N \log(N)).$$

3.4.2 Comparison with other approaches

The computational costs of Algorithm 1 are comparable to the approach by Jäckel (2014, Sec.4), where we incur some additional costs arising from the sorting by liquidity and the effort to retain infeasible quotes. Both filtering procedures are designed directly in terms of a particular feasible solution with intuitive properties. A different approach is taken by Le Floc’h and Oosterlee (2019a, 2019b) and Cohen et al. (2020), in which a particular objective function is specified with constraints, from which the corrected quotes are obtained as its solution. More specifically, preprocessing of the quotes is formulated as a quadratic program (QP) or a linear program (LP), respectively.

For LPs, there are two main approaches to minimization. The first is the simplex method, which often performs well in practice but has a worst-case complexity that is exponential in the number of variables, N . The second approach is the interior-point method, which typically has an $O(N^3)$ complexity per iteration (e.g., Nesterov and Nemirovskii 1994; Nemirovski 2004) and is often faster for large problems (Nocedal and Wright 2006, Ch.14.4; Press, Teukolsky, Vetterling, and Flannery 2007, p.549). This method is also standard for quadratic programming problems (Nocedal & Wright, 2006, Ch.16.6). Given the smaller $O(N \log(N))$ complexity of the proposed filtering method, we may expect that the differences in computational costs will generally become more apparent as the number of quotes, N , increases.

The direct filtering approaches circumvent the costs associated with numerical optimization (e.g., building the constraint matrix and searching for the optimum), making them computationally very light. A compelling property of the optimization-based approaches is that the corrected quotes are optimal with respect to the chosen objective function, which provides a strong theoretical foundation. In the next section, it will be shown that the proposed filtering method can also be characterized as the minimizer of an intuitive objective function, aligning it with the aforementioned approaches.

4 Filter properties

This section presents and discusses several theoretical properties of the arbitrage filtering method in Algorithm 1. Section 4.1 introduces a loss function that aligns closely with how traders view market data. The arbitrage filter is shown to be a minimizer of the

corresponding objective function. Section 4.2 then discusses local filter properties that offer practical insight and guidance regarding the choice of the λ parameter.

Throughout, we assume the availability of N quotes $Q_i = (\mathcal{K}_i, \mathcal{C}_i)$, $i = 1, \dots, N$, for a given expiry date. To economize on notation, regular indices will be used to indicate the ranking by the chosen liquidity proxy (i.e., $\mathcal{C}_i = \mathcal{C}_{(i)}$), with $\mathcal{C}_1$ the most liquid price and $\mathcal{C}_N$ the least. We use $c_\lambda = (\mathcal{C}_{i,\lambda})_{i=1}^N$ to denote the solution from Algorithm 1 for $\lambda \in [0, 1]$.

4.1 Priority loss and arbitrage filter as minimizer

We consider an objective function of the usual additive form

$$f(x) = \sum_{i=1}^N L_i(X_i, \mathcal{C}_i), \quad (12)$$

where $x = (X_i)_{i=1}^N$ is a candidate for the normalized call prices, and the i -th loss term L_i represents the cost of replacing $\mathcal{C}_i$ by X_i . The aim is to minimize the above objective function subject to the no-arbitrage restrictions in (1)–(4), which defines the set of feasible candidates. For this we explore the following *priority loss function*,

$$L_i(X_i, \mathcal{C}_i) = \begin{cases} 0 & \text{if } X_i = \mathcal{C}_i, \\ 2^{N-i} & \text{otherwise.} \end{cases} \quad (13)$$

This loss function ensures that keeping the i -th liquid price unchanged (i.e., $X_i = \mathcal{C}_i$) is preferred over preserving the remaining (less liquid) prices $\mathcal{C}_j$ for $j > i$, as shown formally in Appendix B. To settle ideas, Table 4 shows the resulting loss values based on the example price data and filtered prices in Table 2. The corresponding objective value based on (12) attained by the arbitrage filtering method is $2 + 1 = 3$. An alternative candidate which modifies the third-ranked quote but leaves the others unchanged would attain an objective value of $2^{5-3} = 4$, which is thus inferior (in addition to being infeasible). This is analogous to lexicographical ordering, where earlier positions are prioritized.

In practice it is preferable to consider the normalized priority objective (NPO) value, defined by dividing the objective function by its maximum value:

$$f_{\text{NPO}}(x) = \frac{f(x)}{\sum_{i=1}^N 2^{N-i}} = \frac{\sum_{i=1}^N L_i(X_i, \mathcal{C}_i)}{2^N - 1} = \frac{\sum_{i=1}^N \mathbb{I}(X_i \neq \mathcal{C}_i) 2^{-i}}{1 - 2^{-N}}, \quad (14)$$

where the third expression follows from the partial geometric series, and the final expression is preferred for numerical stability. Contrary to $f(x)$, the NPO takes on values between zero (best; no quotes adjusted) and one (worst; all quotes adjusted), which is

Table 4: Priority loss values based on the example data set and filtered prices in Table 2. The corresponding objective value based on (12) is $2 + 1 = 3$.

Liquidity rank (i)	1	2	3	4	5
Price adjustment	No	No	No	Yes	Yes
Loss L_i	0	0	0	2	1

easier to interpret. For example, the arbitrage filtering method attains an NPO value of $3/31 = 0.097$, which is relatively close to zero. For comparison, an alternative (infeasible) candidate that adjusts only the first-ranked quote would yield an NPO value of $16/31 = 0.516$, illustrating that the priority loss strongly discourages modifications in the most liquid or otherwise prioritized quotes. This is in the same spirit as how traders view market data, where certain key variables (e.g., the ATM quote) are regarded as substantially more important than others.

The following result shows that the objective value attained by the arbitrage filtering method is optimal among the feasible candidates.

Theorem 1. *For any $\lambda \in [0, 1]$, the solution $c_\lambda = (\mathcal{C}_{i,\lambda})_{i=1}^N$ returned by Algorithm 1 is a minimizer of the priority objective function defined by (12) and (13) over the set of feasible candidates that satisfy the arbitrage restrictions in (1)–(4).*

Theorem 1 reflects the design of Algorithm 1, particularly Step 3, which aims to preserve as many of the most liquid quotes as possible. This is done by iterating through the liquidity-sorted quotes, keeping prices unchanged whenever they are consistent with the arbitrage restrictions implied by the previously accepted quotes. The tendency to produce sparse solutions, as opposed to compromising through small perturbations to all prices, is a distinctive feature of the priority loss function. As Cohen et al. (2020, p.353) note, this sparsity is often desirable in the preprocessing of option prices.

4.2 Local filter properties

We now discuss two local filter properties that take into account the specific value of the λ parameter used. The properties are called *local* because they pertain to a particular quote, specifically, the most liquid quote for which two candidates differ.

Our first result considers differences among solutions produced by Algorithm 1.

Proposition 2. *Let $c_{\lambda_1} = (\mathcal{C}_{i,\lambda_1})_{i=1}^N$ and $c_{\lambda_2} = (\mathcal{C}_{i,\lambda_2})_{i=1}^N$ be two different solutions returned by Algorithm 1 for $0 \leq \lambda_1 < \lambda_2 \leq 1$, with m the smallest index such that $\mathcal{C}_{m,\lambda_1} \neq \mathcal{C}_{m,\lambda_2}$. Then we have*

$$|\mathcal{C}_{m,\lambda_1} - \mathcal{C}_m| < |\mathcal{C}_{m,\lambda_2} - \mathcal{C}_m|.$$

This result shows that when two solutions differ, the adjustment to the most liquid price where they differ decreases with λ . In particular, the adjustment is minimized when the parameter is set to zero. The next result demonstrates that the solution with $\lambda = 0$ retains this local optimality property relative to any feasible candidate.

Proposition 3. *Let $c_0 = (\mathcal{C}_{i,0})_{i=1}^N$ be the solution returned by Algorithm 1 with $\lambda = 0$, and suppose $\tilde{c} = (\tilde{\mathcal{C}}_i)_{i=1}^N$ is a different feasible solution, with m the smallest index such that $\tilde{\mathcal{C}}_m \neq \mathcal{C}_{m,0}$. Then we have*

$$|\mathcal{C}_{m,0} - \mathcal{C}_m| < |\tilde{\mathcal{C}}_m - \mathcal{C}_m|.$$

This local optimality property has important practical implications. Specifically, by assigning the first rank to the ATM quote, the solution returned by Algorithm 1 with $\lambda = 0$ ensures that this quote receives the smallest possible adjustment among *all* feasible candidates for the call prices.

Taken together, the results above provide a clear recommendation regarding the λ parameter: it should be set as small as possible, which means zero, unless application-specific considerations require a positive value. For example, if strict monotonicity or convexity is needed, a value of $\lambda > 0$ should be used, with Proposition 2 suggesting the minimum value appropriate for the application.

The above results are local, hence they may not always extend to all other adjusted quotes (i.e., globally). Importantly, however, they pertain to the most liquid quote adjusted, which is generally more important than any of the remaining quotes in isolation. In addition, there are various situations in which the smallest possible local adjustment directly implies global optimality. This is trivially true when only a single quote is adjusted, but more generally, it also applies when adjusting one quote does not affect the bounds for subsequent quotes. Figure 1 provides an example of this, in which the adjustment to the fourth-ranked quote has no impact on the feasible price range for the fifth-ranked quote, illustrating global optimality of minimal local adjustments.

5 Filtering in the expiry dimension

If option quotes are available for multiple expiries, a straightforward approach is to preprocess them separately for each expiry. This resolves common data errors in practical applications, where issues often arise from stale or inaccurate implied volatility quotes at extreme strikes (e.g., Jäckel, 2014, Sec.4). For situations where addressing arbitrage opportunities in the expiry dimension is important, this section introduces an extension of the filtering method. This procedure filters quotes one expiry at a time in increasing order. The inequality in (7) is considered locally, by comparing quotes with the lower bounds implied by previous expiries. We refer to Cohen et al. (2020) for a global alternative to preprocessing under sufficient conditions for the absence of static arbitrage on the call price surface. The method presented below integrates naturally with Algorithm 1.

Suppose we have available quotes for increasing expiries T_j , $j = 1, \dots, N_T$, for some number of expiries $N_T > 1$. The idea is to start at T_1 by applying the filtering procedure from Section 3.1, and to iteratively filter the next expiries with a modified lower bound. In particular, for $j = 2, \dots, N_T$, we apply Algorithm 1 with $T = T_j$ and the lower bound $\underline{\mathcal{C}}(\mathcal{K}, T)$ replaced by the infimum

$$\mathcal{C}_{\text{Inf}}(\mathcal{K}, T) = \max_{k=1, \dots, j} \underline{\mathcal{C}}(\mathcal{K}, T_k). \quad (15)$$

The above condition follows because $\mathcal{C}(\mathcal{K}, T) = \bar{\mathcal{C}}(\mathcal{K}, T)$ would hold if $\mathcal{C}(\mathcal{K}, T)$ were accepted, so that (7) implies $\mathcal{C}(\mathcal{K}, T) \geq \mathcal{C}_{\text{Inf}}(\mathcal{K}, T)$. Notably, by proceeding in order of increasing expiry, the prices and bounds implied by earlier expiries will not change after a quote at the current expiry has been accepted, ensuring that the condition in (7) remains satisfied for the quote considered. It also simplifies matters, as we need only consider one bound from other expiries, rather than two.

Figure 2 shows the result of applying the extended filtering method to the option price surface composed of the example data in Tables 2 and 5. The filtering procedure is initialized by applying Algorithm 1 to the option prices at expiry $T_1 = 1$. The leftmost plot in the top row in Figure 2 shows that the data set is not consistent with an arbitrage-free call price function, as the two rightmost normalized call prices at expiry $T_2 = 2$

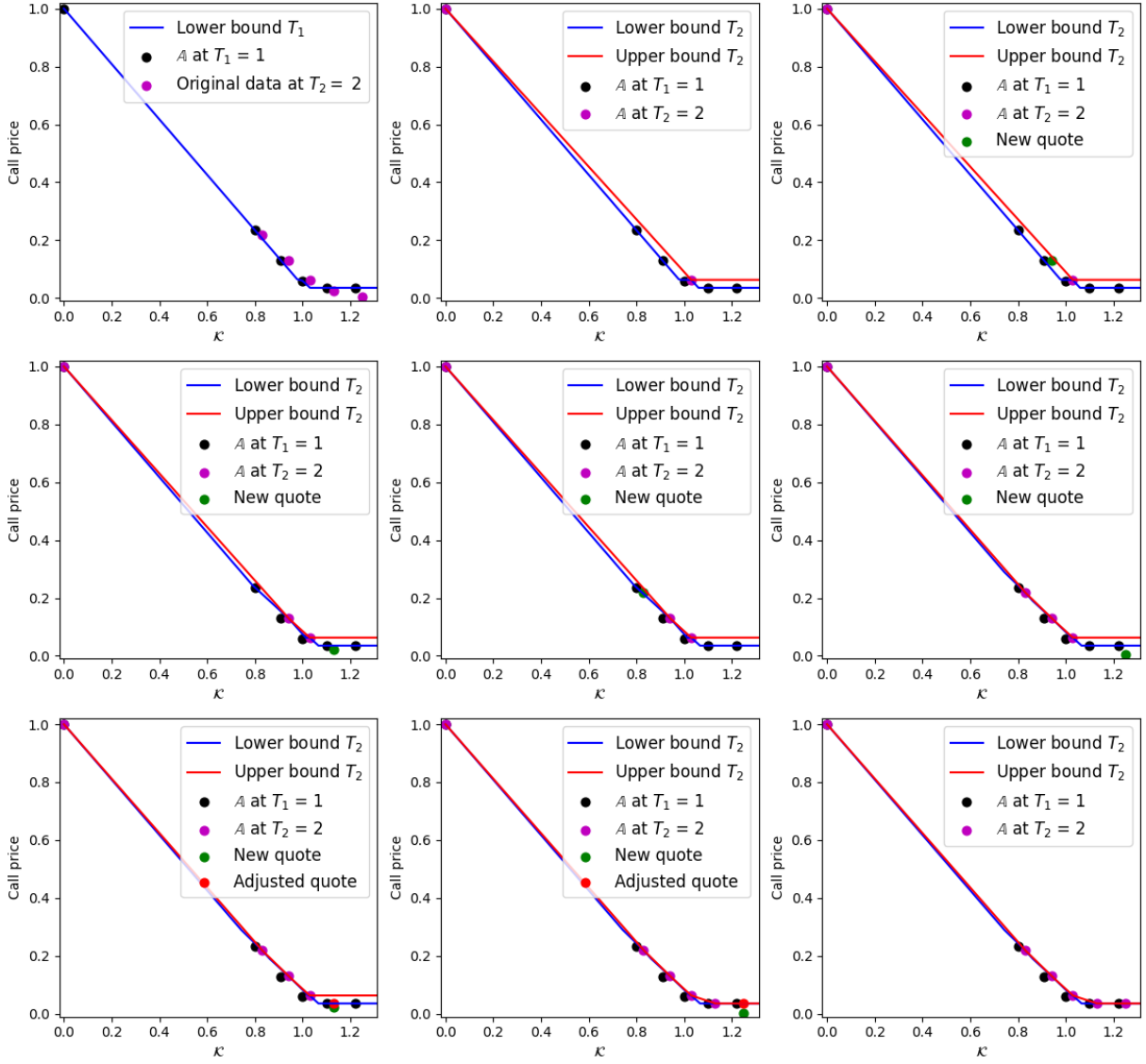


Figure 2: Extended arbitrage filtering method from Section 5 illustrated step-by-step for the example data from Tables 2 and 5.

are below the lower bound implied by the filtered option price data at expiry T_1 . The remaining plots illustrate how the information from the previously filtered option prices is used to check and adjust the data for expiry T_2 .

As noted above, the approach is local in that the condition in (7) is considered at

Table 5: Original and filtered example data, with $T_2 = 2$ and $F(T_2) = B(T_2) = 1$. In this case, $\mathcal{K} = K$ and $\mathcal{C}(\mathcal{K}, T) = C(K, T)$.

Moneyiness	0.83	0.94	1.03	1.13	1.25
Liquidity rank (i)	4	2	1	3	5
Call price (original)	0.22	0.13	0.06	0.02	0.00
Call price (filtered)	0.22	0.13	0.06	<i>0.04</i>	<i>0.04</i>

the moneyness of the current quotes, while we may expect the same will typically hold at intermediate values, where the upper bound is defined by linear interpolation of the accepted quotes. If violated at the exact location where a quote is to be adjusted in Step 4 of the algorithm, the usual lower bound from Section 3.2 can be applied as a fallback, though this situation has not been observed in practice.

As the example illustrates, the iterative nature of the extension means that quotes from later expiries are more likely to be adjusted compared to separate applications of the approach from Section 3.1. Depending on the context, it may be more suitable to initialize the method at a later expiry $T_j > T_1$. For instance, when the options corresponding to T_1 (e.g., the one day expiry) are known to have low trading volume relative to the next expiry. The previous expiries could then be filtered in reverse order, using the supremum defined analogously to (15).

6 Numerical examples

In this section we consider numerical examples based on real data sets to illustrate and discuss key properties of the proposed arbitrage filtering method.

6.1 Filtered quotes and implied risk-neutral distribution

We consider option price data consisting of 76 settlement implied volatility quotes for the Deutscher Aktienindex (DAX) with three days to expiry from the EUREX exchange on June 13, 2000. The bond price is computed using a zero rate of 4.36%, and the forward is equal to 7271.52. The dividend yield is assumed to be zero because the DAX is a performance index. The option prices, taken from Appendix B of Fengler (2009), are shown in Figure 3.

The top and middle plots show the implied volatilities and call prices, respectively. The bottom plot focuses on the call prices for a subset of the domain, which reveals several violations of monotonicity and convexity. The plots also show the results of applying Algorithm 1 to the DAX data. In the filtered data, the arbitrage violations are resolved. The first 18 quotes having strike closest to the forward were accepted, with the majority of the violating quotes being far in or out-of-the-money options. In total, 48 quotes (63.2%) were adjusted, illustrating the filter’s tendency to produce sparse solutions. As shown in Figure 3, the adjustments are typically minor.

For further analysis, we consider the risk-neutral cumulative distribution function (CDF) computed via

$$\mathbb{Q}(S_T \leq s) = 1 + \frac{\partial C / \partial K|_{K=s}}{B(T)}. \quad (16)$$

Figure 4 shows the risk-neutral CDFs corresponding to linear interpolation of the original and filtered call prices. As emphasized in the two plots on the right, linear interpolation of the original quotes does not result in a valid CDF, since the graph is decreasing at several locations. While the call price function obtained from linear interpolation may not be realistic, the mean value theorem implies a non-monotonic CDF for *any* differentiable call price function that interpolates the original quotes. On the other hand, the risk-neutral CDF based on the filtered quotes is seen to be monotonic.

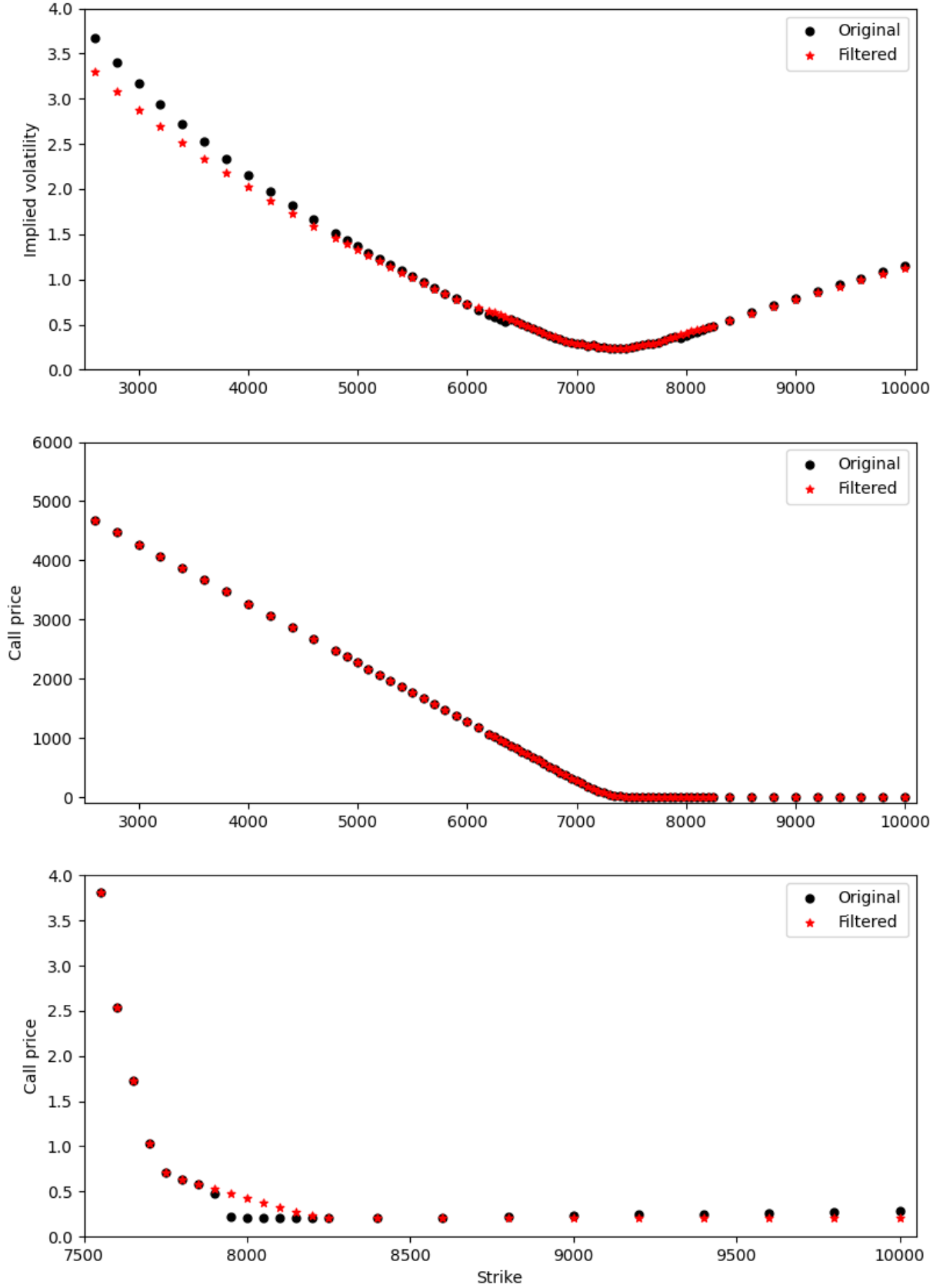


Figure 3: Original and filtered DAX three day settlement option quotes from the EUREX exchange on June 13, 2000, with $F(T) = 7271.52$, $B(T) = \exp(-4.36/100 \cdot T)$ and $T = 3/365$. The filtered quotes are based on Algorithm 1 with $\lambda = 0$.

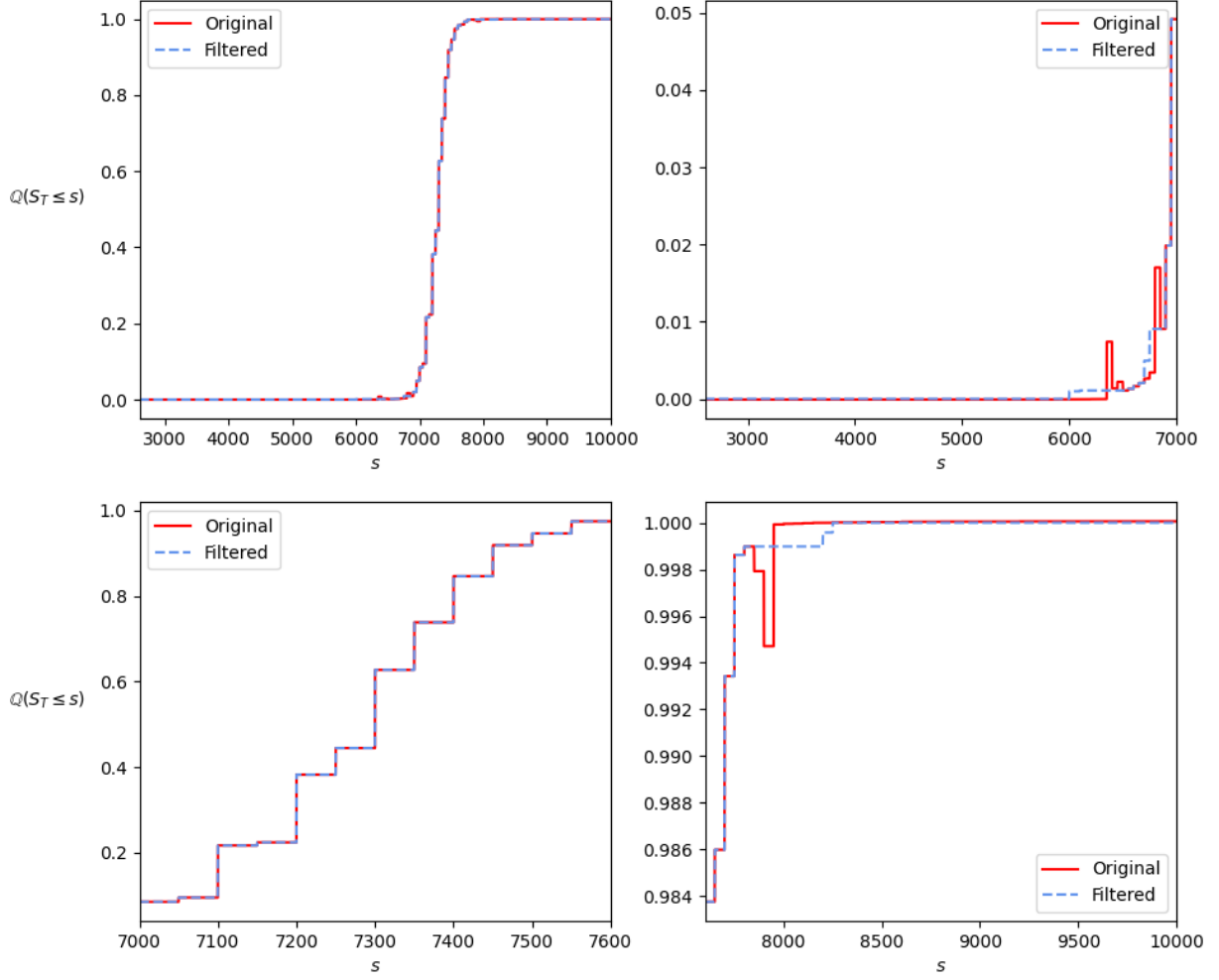


Figure 4: Risk-neutral cumulative distribution functions (CDFs) for the DAX option price data set based on (16) with linear interpolation of the original and filtered quotes. The risk-neutral CDFs are plotted using a continuous line to emphasize (non-)monotonicity.

6.2 Interpolation of filtered quotes

6.2.1 Smooth interpolation

One of the main uses of option price data is to serve as input for the calibration of derivative valuation models. Such models are needed to determine prices for a continuum of values for the contract parameters, such as the strike price. In practice, a common approach is to interpolate the given option prices for a particular expiry along the strike or moneyness dimension (e.g., Rebonato 2005; Homescu 2011). However, when the data are subject to arbitrage opportunities, a direct interpolation of option quotes will not provide sensible prices. This strategy may be improved upon by first applying the proposed arbitrage filtering method.

As illustration we consider the Tesla implied volatility quotes on June 15, 2018 with expiry $T = 1.592$ taken from Le Floc’h and Oosterlee (2019b, Appendix A). Figure 5 shows the quotes together with the result of a naive direct interpolation (the red, solid line). The interpolation is based on the well-known piecewise cubic method of Fritsch

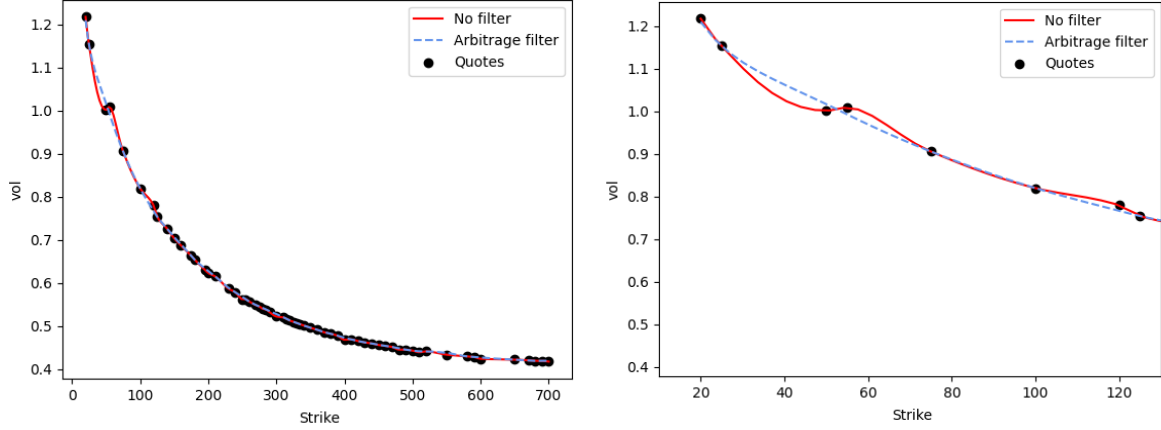


Figure 5: Tesla implied volatility quotes on June 15, 2018 with expiry $T = 1.592$ and comparison of interpolations: naive direct interpolation of implied volatilities, as well as their interpolation after applying Algorithm 1 with $\lambda = 0.1$.

and Butland (1984) in terms of implied volatilities, with $\log(K)$ as coordinate. The direct interpolation provides an exact match to each quote, which results in a counterintuitive shape at various locations in the implied volatility smile. The figure also shows the same interpolation method, but now after applying Algorithm 1 (the blue, dashed line). The interpolation based on the arbitrage filtering method does not match all the quotes, which indicates several arbitrage violations in the strike dimension. Notably, the odd shape is no longer present in the interpolated prices.

The chosen interpolation method and configuration is but one of many possible choices, but a direct interpolation of the prices will always retain arbitrage opportunities that are present in the data. The direct interpolation approach remains popular in practice, as it is simple and fast, where the latter is a crucial requirement in online settings. An important advantage of the proposed filtering method is that it retains these properties. Filtering the option quotes before interpolation can therefore be important in real-time pricing applications.

If the quotes serve as inputs for a *smooth* interpolation scheme, a further consideration is that the parameter λ can have an effect on the curvature of the function that interpolates the prices. As explained in Section 3.2, the bounds used to check and adjust the quotes are determined by linear inter and extrapolation. In particular, setting an adjusted value exactly equal to a bound, as with $\lambda = 0$ or $\lambda = 1$, may result in placing three quotes on a straight line, which could pose a challenge for smooth interpolants. The above can simply be resolved by setting the parameter to a suitable value in the interval $(0, 1)$, such as $\lambda = 0.1$, say. Since the infeasible quotes are adjusted after checking and potentially adding all other quotes, the feasible price range is usually small. The results therefore typically show low sensitivity to the choice of λ . This property is illustrated in Table 6, which shows the MAE and RMSE in implied volatility times 100 for the Tesla data set attained by the arbitrage filtering method in Algorithm 1 for various values of the λ parameter. As expected, the pricing errors increase with the value of λ , but its overall impact remains relatively small, especially for values below 0.2.

Table 6: Mean absolute error (MAE) and root mean squared error (RMSE) in implied volatility times 100 for the Tesla data set attained by the arbitrage filtering method in Algorithm 1 for various values of the λ parameter.

λ	0.01	0.02	0.05	0.1	0.2	0.5
MAE	0.159	0.160	0.163	0.168	0.178	0.208
RMSE	0.377	0.378	0.380	0.386	0.404	0.489

6.2.2 Comparison with exclusion filtering

We compare the performance of the proposed method with an exclusion filtering approach, in which violating quotes are discarded. The exclusion filter is implemented by omitting Step 4 from Algorithm 1, so that only the directly accepted quotes are retained. From a valuation perspective, we can expect that the use of exclusion filters results in interpolated prices that are further from the original data.

To illustrate this, we use the above exclusion filter to interpolate the accepted implied volatilities for the Tesla data set. As the results depend on the interpolation method, we consider several alternatives: linear interpolation, a natural cubic spline (NCS), a clamped cubic spline (CCS), and a piecewise monotonic cubic spline (PCHIP; Fritsch & Butland, 1984). Strictly speaking, while such approaches ensure consistency with the European option bounds in (4) through the Black (1976) formula, they may not preserve the other shape constraints in between the quotes. In practice, however, volatility interpolation is often found to yield sensible and intuitive prices when applied to correct input data.

Table 7 shows the resulting MAE and RMSE in implied volatility, multiplied by 100. A comparison with Table 6 shows that none of the fits match the performance of Algorithm 1 applied with $\lambda = 0.2$. Instead, the results are largely comparable with those attained by Algorithm 1 applied with $\lambda = 0.5$, which places the adjusted quote in the middle of the feasible call price range and essentially ignores the information from the violating quote. Discarding quotes, at the very least, also discards information about *which* bounds were violated. This information is retained in the proposed arbitrage filtering method.

Table 7: Mean absolute error (MAE) and root mean squared error (RMSE) in implied volatility multiplied by 100 for the Tesla data set, based on the exclusion filter defined by Algorithm 1 without Step 4. The accepted implied volatilities are interpolated using the following methods: linear interpolation, a natural cubic spline (NCS), a clamped cubic spline (CCS), and a piecewise monotonic cubic spline (PCHIP; Fritsch & Butland, 1984).

Interpolant	Linear	NCS	CCS	PCHIP
MAE	0.214	0.207	0.188	0.201
RMSE	0.607	0.521	0.474	0.530

6.3 Optimal filtering

6.3.1 Comparison of optimal filters

This section provides a comparison of the arbitrage filtering method in Algorithm 1 with the solutions that minimize the (R)MSE and MAE in terms of call prices (Le Floc’h and

Oosterlee 2019b; Cohen et al. 2020), using the Tesla and DAX data sets from Sections 6.1 and 6.2 for illustration. For the optimal MSE solution, we use the QP approach from Le Floc’h and Oosterlee 2019b. For the optimal MAE solution, we apply the auxiliary variable technique and extend the constraint matrix accordingly to obtain the corresponding LP. Both programs are solved using interior-point methods, relying on the `cvxopt` package, which implements its core algorithms in the C language for high performance. The arbitrage filter is applied with $\lambda = 0$.

Table 8 presents the results for both data sets; for readability, the MAE, RMSE, and NPO values have been multiplied by 100. The MAE and RMSE values pertain to adjustments to the original quotes in terms of implied volatility. For computing the NPO values, a threshold of $\epsilon = 10^{-7}$ in volatility difference is used to indicate a price adjustment. This tolerance excludes spurious adjustments arising solely from finite-precision floating-point arithmetic. For high precision, we rely on implementations of the Black option price function and implied volatility solver as in Jäckel (2015), which ensures these conversions do not affect the computed NPO values.

For the Tesla data set, the MAE and RMSE values across the different methods are relatively close, with the LP and QP approaches performing best for their respective objective functions. As indicated by the NPO value, the arbitrage filter makes fewer changes to the quotes with the highest liquidity proxy, resulting in about half the value of the second-best NPO. For the DAX data set, the MAE and RMSE values increase for all methods. This reflects the larger average size and dispersion of the quotes: the sample mean and standard deviation of the implied vols are 0.877 and 0.806, respectively, compared to 0.569 and 0.177 for the Tesla data set. For the arbitrage filter, the MAE and RMSE values are now somewhat larger relative to the other methods, but in absolute terms they remain small (recalling that all values are multiplied by 100). Its NPO value is zero after rounding, as the first 18 quotes were accepted in Step 3 of Algorithm 1.

Notably, the minimum MAE value for the DAX data set is attained by the QP, rather than the LP. These approaches are designed to fit call prices, for which the LP indeed attains a lower MAE value of 3.470, compared to 3.788 for the QP (where both have been multiplied by 100). However, since implied volatility is a nonlinear transformation

Table 8: Comparison of optimal filters: objective values for various loss functions based on implied volatility quote adjustments in the Tesla and DAX data sets (see Sections 6.1 and 6.2). For computing the normalized priority objective (NPO), a threshold of $\epsilon = 10^{-7}$ in volatility difference is used to indicate a price adjustment, which serves to exclude spurious adjustments arising solely from finite-precision floating-point arithmetic. Results are shown for the arbitrage filtering (AF) procedure in Algorithm 1, along with linear programming (LP) and quadratic programming (QP) approaches for comparison. For readability, all objective values have been multiplied by 100. Running times are reported in milliseconds and represent averages over 1000 repetitions performed on a computer with a Ryzen 9 5950X processor.

Data Set	Tesla ($N = 61$)				DAX ($N = 76$)			
Method	MAE	RMSE	NPO	Time	MAE	RMSE	NPO	Time
LP	0.115	0.371	89.8	12.4	1.327	2.761	1.5	19.4
QP	0.134	0.317	25.0	3.8	1.246	2.376	17.2	8.0
AF	0.158	0.376	12.5	1.1	3.923	8.611	0.0	1.5

of the call price, optimality in one domain does not necessarily translate to optimality in the other. This differs for the NPO, which, due to the binary aspect of the priority loss function, is invariant under strictly monotone transformations of the quotes, such as the conversion from call prices to implied volatility (or vice versa). Consequently, this invariance also applies to its set of minimizers.

Lastly, Table 8 also shows the running times in milliseconds, averaged over 1000 repetitions performed on a computer with a Ryzen 9 5950X processor. For both data sets, the arbitrage filtering method runs in under 2 milliseconds on average. Notably, this is based on an implementation in `python`, illustrating the computationally light nature of the algorithm and leaving room for further reductions through compiled languages. The difference in running time relative to the optimization approaches is most pronounced for the DAX data set, which contains more quotes ($N = 76$ vs $N = 61$), in line with the computational complexities discussed in Section 3.4.

6.3.2 Robust filtering

In practice we may encounter data sets characterized by outliers. It can then be important to ensure that the results are not disproportionately impacted by such extreme observations. To illustrate how the arbitrage filtering method deals with such cases, we perturb one of the quotes in the DAX data set from Section 6.1. This is done by multiplying the smallest implied volatility (0.236), corresponding to strike $K = 7400$, by a factor of 100. This emulates an example of incorrect data reading or misreporting, known to occur with unfortunate frequency and impact in practice. Figure 6 shows the perturbed data set in terms of implied volatility (top) and call prices (bottom).

The figure also includes the adjusted quotes based on the QP approach and the arbitrage filtering method. The quotes returned by the QP indicate a substantial impact from the perturbed quote. This can be understood from the MSE objective, in which its squared error has a large influence. By contrast, the proposed arbitrage filter is essentially unaffected by the perturbation of the data. This robustness follows from Steps 3 and 4 of Algorithm 1, which show that the size of the error becomes irrelevant once the quote is characterized as infeasible, as the adjusted value is determined solely by the previously accepted quotes and whether the lower or upper bound was violated. This insensitivity to the size of the error can also be understood from the priority objective function, where only the rank and matching of the quotes matter.

7 Discussion and conclusion

7.1 Discussion

This section briefly discusses some further possible extensions of the proposed method. The filtering procedure could, in principle, be applied to any option type that requires prices to be monotone and convex in the strike parameter, and for which model-free lower and upper bounds, as well as the price for strike zero, are known. Potential applications include caps and floors, swaptions, as well as American and Asian options. In such cases, sufficient no-arbitrage conditions may be unavailable, but the proposed approach at the very least ensures that no unnecessary changes are introduced when the data are correct.

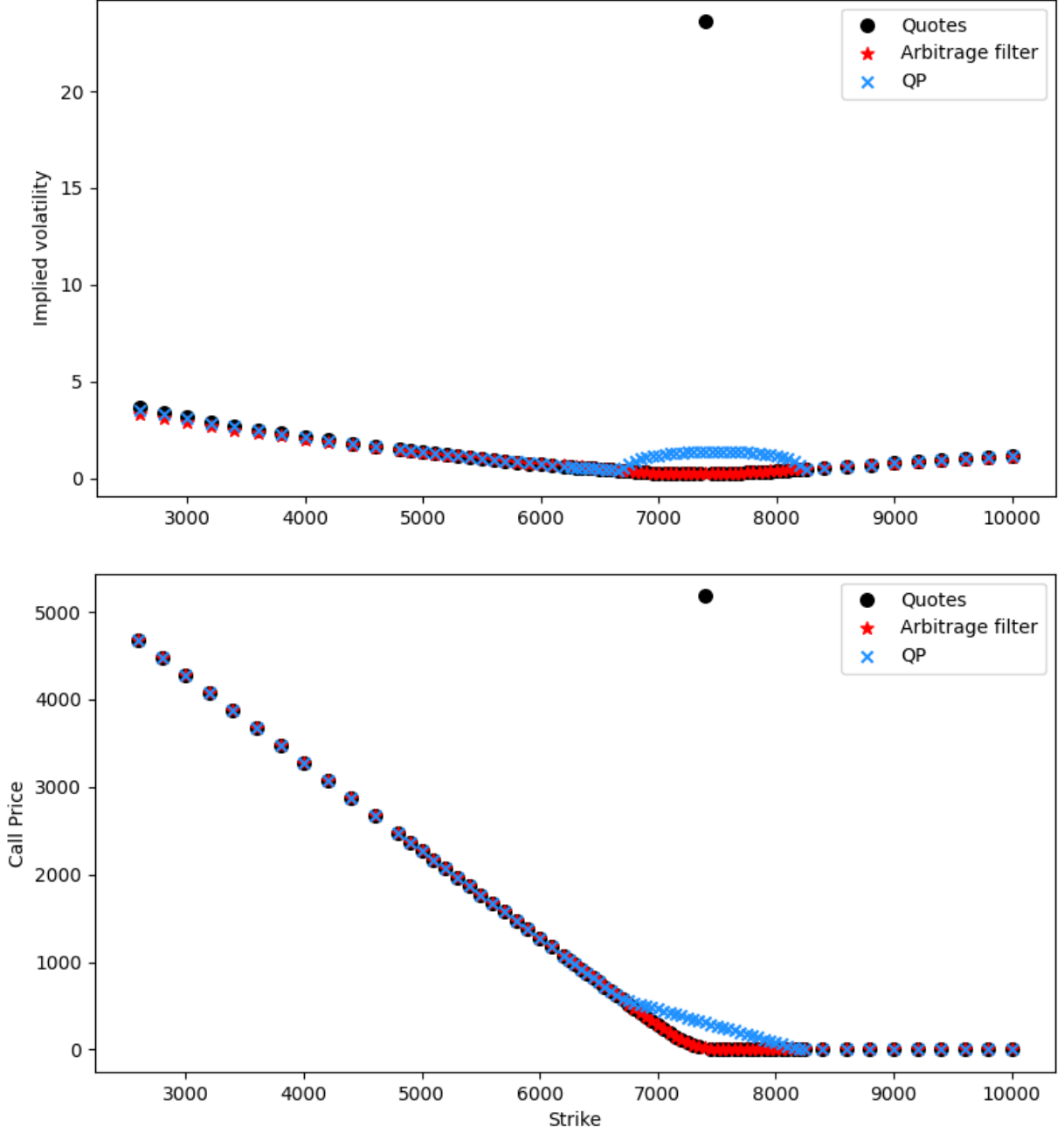


Figure 6: Sensitivity analysis based on perturbed DAX option prices from Section 6.1: the smallest implied volatility, corresponding to strike $K = 7400$, is multiplied by 100. The implied volatility and call prices are shown, as well as their processed values based on the arbitrage filtering method in Algorithm 1 with $\lambda = 0$ and the quadratic programming (QP) approach.

As discussed previously, the use of $\lambda > 0$ allows for enforcing strict shape constraints on the rejected quotes. In cases where it is desirable to impose strict constraints on *all* quotes, Step 3 in Algorithm 1 can be adapted to use the modified bounds $\underline{\mathcal{C}}^* = \underline{\mathcal{C}} + \lambda(\bar{\mathcal{C}} - \underline{\mathcal{C}})$ and $\bar{\mathcal{C}}^* = \bar{\mathcal{C}} + (1 - \lambda)(\bar{\mathcal{C}} - \underline{\mathcal{C}})$ based on (10), where $\lambda \in (0, 0.5)$ is assumed to prevent crossing of the bounds. Given the definition of the infimum in (15), this also means that the strict version of the expiry condition in (7) is considered in the extended filtering procedure from Section 5.

7.2 Conclusion

This paper presents a simple arbitrage filtering method for preprocessing European option quotes. Prices are modified only when basic no-arbitrage conditions are violated, and no quotes are discarded. Instead, quotes are adjusted using information from other, more liquid quotes. The adjustments are designed to align closely with how traders view market data, where certain key variables, such as the ATM quote, are regarded as substantially more important than others. The method is straightforward to implement and fast, making it particularly suitable for real-time applications. Key filtering properties are illustrated through numerical examples, including the tendency to produce sparse solutions that leave many quotes unchanged, as well as its robustness to outliers. Beyond its general use as a preprocessing technique, the filter can enhance standard interpolation methods and offer a solution for other approaches requiring arbitrage-free quotes as input.

Acknowledgments

I am grateful to Matteo Michielon, Raoul Pietersz, and Mark-Jan Boes for their comments on an earlier version of this paper. I also thank the handling editor and two anonymous referees for their thoughtful feedback, which greatly helped to improve the paper.

Funding details

No funding was received.

Disclosure statement

The author reports there are no competing interests to declare.

Data availability statement

The DAX data set is given in Appendix B of Fengler (2009) and the Tesla data set in Appendix A of Le Floc’h and Oosterlee (2019b).

References

- Bakshi, G., Cao, C., & Chen, Z. (1997). Empirical performance of alternative option pricing models. *The Journal of finance*, 52(5), 2003–2049.
- Bams, D., Blanchard, G., & Lehnert, T. (2015). On the impact of exclusion filters rules in option pricing. *Unpublished manuscript*, 77(4), 835–874.
- Barone-Adesi, G., Engle, R. F., & Mancini, L. (2008). A garch option pricing model with filtered historical simulation. *The review of financial studies*, 21(3), 1223–1258.
- Black, F. (1976). The pricing of commodity contracts. *Journal of financial economics*, 3(1-2), 167–179.

- Bliss, R. R., & Panigirtzoglou, N. (2004). Option-implied risk aversion estimates. *The journal of finance*, 59(1), 407–446.
- Bollen, N. P., & Whaley, R. E. (2004). Does net buying pressure affect the shape of implied volatility functions? *The Journal of Finance*, 59(2), 711–753.
- Brandt, M. W., & Wu, T. (2002). Cross-sectional tests of deterministic volatility functions. *Journal of Empirical Finance*, 9(5), 525–550.
- Cohen, S. N., Reisinger, C., & Wang, S. (2020). Detecting and repairing arbitrage in traded option prices. *Applied Mathematical Finance*, 27(5), 345–373.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT press.
- Cousot, L. (2007). Conditions on option prices for absence of arbitrage and exact calibration. *Journal of Banking & Finance*, 31(11), 3377–3397.
- Davis, M. H., & Hobson, D. G. (2007). The range of traded option prices. *Mathematical Finance*, 17(1), 1–14.
- Embrechts, P., Frey, R., & McNeil, A. (2015). *Quantitative risk management*.
- Fengler, M. R. (2009). Arbitrage-free smoothing of the implied volatility surface. *Quantitative Finance*, 9(4), 417–428.
- Foucault, T., Pagano, M., & Röell, A. (2023). *Market liquidity: theory, evidence, and policy* (2nd ed.). Oxford University Press.
- Fritsch, F. N., & Butland, J. (1984). A method for constructing local monotone piecewise cubic interpolants. *SIAM journal on scientific and statistical computing*, 5(2), 300–304.
- Gatheral, J. (2004). A parsimonious arbitrage-free implied volatility parameterization with application to the valuation of volatility derivatives. *Presentation at Global Derivatives & Risk Management, Madrid*.
- Gatheral, J. (2011). *The volatility surface: a practitioner’s guide*. John Wiley & Sons.
- Haven, E., Liu, X., & Shen, L. (2012). De-noising option prices with the wavelet method. *European Journal of Operational Research*, 222(1), 104–112.
- Homescu, C. (2011). Implied volatility surface: Construction methodologies and characteristics. *arXiv preprint arXiv:1107.1834*.
- Ivanovas, A., & Meier, P. (2015). On working with option data: implications of data choice. *PhD Thesis*.
- Jäckel, P. (2014). Clamping down on arbitrage. *Wilmott*, 71(2014), 54–69.
- Jäckel, P. (2015). Let’s be rational. *Wilmott*, 2015(75), 40–53.
- Kahalé, N. (2004). An arbitrage-free interpolation of volatilities. *Risk*, 17(5), 102–106.
- Koopman, S. J., Jungbacker, B., & Hol, E. (2005). Forecasting daily variability of the s&p 100 stock index using historical, realised and implied volatility measurements. *Journal of Empirical Finance*, 12(3), 445–475.
- Le, H., & Gregoriou, A. (2020). How do you capture liquidity? a review of the literature on low-frequency stock liquidity. *Journal of Economic surveys*, 34(5), 1170–1186.
- Le Floc’h, F., & Oosterlee, C. W. (2019a). Model-free stochastic collocation for an arbitrage-free implied volatility: Part i. *Decisions in Economics and Finance*, 42(2), 679–714.
- Le Floc’h, F., & Oosterlee, C. W. (2019b). Model-free stochastic collocation for an arbitrage-free implied volatility, part ii. *Risks*, 7(1), 30.
- Malz, A. M. (1997). *Option-implied probability distributions and currency excess returns*

- (Tech. Rep.). Federal Reserve Bank of New York.
- Malz, A. M. (2014). *A simple and reliable way to compute option-based risk-neutral distributions* (Tech. Rep.). Staff report.
- Merton, R. C. (1973). Theory of rational option pricing. *The Bell Journal of economics and management science*, 141–183.
- Nemirovski, A. (2004). *Interior point polynomial time methods in convex programming*. Lecture notes, Georgia Institute of Technology.
- Nesterov, Y., & Nemirovskii, A. (1994). *Interior-point polynomial algorithms in convex programming*. SIAM.
- Neumann, M., & Skiadopoulos, G. (2013). Predictable dynamics in higher-order risk-neutral moments: Evidence from the s&p 500 options. *Journal of Financial and Quantitative Analysis*, 48(3), 947–977.
- Nocedal, J., & Wright, S. (2006). *Numerical optimization*. Springer Science & Business Media.
- Press, W. H., Teukolsky, S. A., Vetterling, W. T., & Flannery, B. P. (2007). *Numerical recipes 3rd edition: The art of scientific computing* (3rd ed.). Cambridge university press.
- Rebonato, R. (2005). *Volatility and correlation: the perfect hedger and the fox*. John Wiley & Sons.
- Schoutens, W. (2003). *Lévy processes in finance: pricing financial derivatives*. Wiley Online Library.
- Wang, Y., Yin, H., & Qi, L. (2004). No-arbitrage interpolation of the option price function and its reformulation. *Journal of Optimization Theory and Applications*, 120(3), 627–649.

A Proof of Proposition 1

We proceed by contradiction. Suppose that $\underline{\mathcal{C}}(\mathcal{K}, T_1) > \bar{\mathcal{C}}(\mathcal{K}, T_2)$. The conditions in (1)–(4) imply that, if $\mathcal{C}(\mathcal{K}, T)$ is arbitrage-free and matches the quotes at T_1 and T_2 , both $\mathcal{C}(\mathcal{K}, T_1) \geq \underline{\mathcal{C}}(\mathcal{K}, T_1)$ and $\mathcal{C}(\mathcal{K}, T_2) \leq \bar{\mathcal{C}}(\mathcal{K}, T_2)$ must hold. Combining the above yields

$$\mathcal{C}(\mathcal{K}, T_1) \geq \underline{\mathcal{C}}(\mathcal{K}, T_1) > \bar{\mathcal{C}}(\mathcal{K}, T_2) \geq \mathcal{C}(\mathcal{K}, T_2).$$

However, this contradicts the expiry monotonicity condition in (6), which requires that

$$\mathcal{C}(\mathcal{K}, T_1) \leq \mathcal{C}(\mathcal{K}, T_2).$$

Hence, there exists a calendar spread arbitrage opportunity on the call price surface $\mathcal{C}(\mathcal{K}, T)$.

B Derivation of the priority property

In this section we show that the loss function in (13) prioritizes keeping the i -th ranked price unchanged over preserving all remaining prices, with $i < N$ assumed to ensure that

at least one subsequent quote exists. This means that

$$L_i(a, b) > \sum_{j=i+1}^N L_j(a, b) \quad (\text{B.1})$$

for arbitrary but distinct real numbers $a \neq b$, used to indicate that all quotes are adjusted. To show this, we start by noting that

$$\sum_{j=i+1}^N L_j(a, b) = \sum_{j=i+1}^N 2^{N-j} = \sum_{m=0}^{N-(i+1)} 2^m = \frac{2^{N-i} - 1}{2 - 1} = 2^{N-i} - 1,$$

where the fraction follows from the partial geometric series. The result in (B.1) then follows because $L_i(a, b) = 2^{N-i} > 2^{N-i} - 1$.

C Proof of Theorem 1

We start by noting that the solution c_λ from Algorithm 1 is feasible for any $\lambda \in [0, 1]$, as it satisfies the no-arbitrage conditions in (1)–(4) by construction. Next, suppose (by contradiction) that there exists an alternative feasible solution, $\tilde{c} = (\tilde{c}_i)_{i=1}^N$, with a lower objective value. There must then be at least one index $i \in \{1, \dots, N\}$ such that

$$\tilde{c}_i = c_i \neq c_{i,\lambda}.$$

Let m be the smallest index for which the above holds. The priority loss implies that, for any $j < m$,

$$c_{j,\lambda} = c_j \implies \tilde{c}_j = c_j.$$

Therefore, the alternative solution $\tilde{c}$ must satisfy all arbitrage restrictions implied by the arbitrage-free set at the start of the m -th iteration of Step 3 in Algorithm 1. Denote this set by $\mathbb{A}_m$. Since $Q_m = (\mathcal{K}_m, \mathcal{C}_m)$ was not added to $\mathbb{A}_m$, it must be that $\tilde{\mathcal{C}}_m = \mathcal{C}_m$ violates at least one of these restrictions. Thus, $\tilde{c}$ is not feasible, yielding a contradiction.

D Proof of Proposition 2

By construction, the two solutions can only differ in the adjusted quotes, determined in Step 4 of Algorithm 1. Since m is the smallest index for which the two solutions differ, the arbitrage-free set is the same under both λ_1 and λ_2 at the start of the m -th iteration in Step 4. This implies that $\mathcal{C}_{m,\lambda_1}$ and $\mathcal{C}_{m,\lambda_2}$ are based on (10) with identical values for the bounds $\underline{\mathcal{C}}$ and $\overline{\mathcal{C}}$. Furthermore, since the m -th quote was not added to the arbitrage-free set in Step 3, it follows that, with respect to these bounds, either $\mathcal{C}_m > \overline{\mathcal{C}}$ or $\mathcal{C}_m < \underline{\mathcal{C}}$. Assuming the latter (without loss of generality), we obtain

$$\mathcal{C}_m < \underline{\mathcal{C}} \leq \mathcal{C}_{m,\lambda_1} = \underline{\mathcal{C}} + \lambda_1 (\overline{\mathcal{C}} - \underline{\mathcal{C}}) < \underline{\mathcal{C}} + \lambda_2 (\overline{\mathcal{C}} - \underline{\mathcal{C}}) = \mathcal{C}_{m,\lambda_2},$$

where the rightmost inequality follows from the fact that $0 \leq \lambda_1 < \lambda_2 \leq 1$, and that $\overline{\mathcal{C}} - \underline{\mathcal{C}} > 0$, which holds because $\mathcal{C}_{m,\lambda_1} \neq \mathcal{C}_{m,\lambda_2}$ by assumption. This shows that $\mathcal{C}_{m,\lambda_1}$ is closer to $\mathcal{C}_m$ than $\mathcal{C}_{m,\lambda_2}$ is, which proves the statement.

E Proof of Proposition 3

If $\mathcal{C}_{m,0} = \mathcal{C}_m$, the statement trivially holds, as $\tilde{\mathcal{C}}_m \neq \mathcal{C}_{m,0}$ by assumption. We therefore consider the case where $\mathcal{C}_{m,0} \neq \mathcal{C}_m$. Since m is the smallest index for which the two candidates differ, it holds that, for any $j < m$,

$$\mathcal{C}_{j,0} = \tilde{\mathcal{C}}_j.$$

Therefore, the alternative candidate $\tilde{c}$ must satisfy all arbitrage restrictions implied by the arbitrage-free set at the start of the m -th iteration of Step 3 in Algorithm 1. Denote this set by $\mathbb{A}_m$, with $\underline{\mathcal{C}}_m = \underline{\mathcal{C}}(\mathcal{K}_m)$ and $\bar{\mathcal{C}}_m = \bar{\mathcal{C}}(\mathcal{K}_m)$ the corresponding bounds implied by the arbitrage restrictions. Since the alternative candidate is feasible, the above implies

$$\underline{\mathcal{C}}_m \leq \tilde{\mathcal{C}}_m \leq \bar{\mathcal{C}}_m. \quad (\text{E.1})$$

Furthermore, since the m -th quote was not added to $\mathbb{A}_m$ in Step 3, it follows that either $\mathcal{C}_m > \bar{\mathcal{C}}_m$ or $\mathcal{C}_m < \underline{\mathcal{C}}_m$. Assuming the latter (without loss of generality), the use of (10) with $\lambda = 0$ implies that $\mathcal{C}_{m,0} = \underline{\mathcal{C}}_m$, so that

$$\mathcal{C}_m < \underline{\mathcal{C}}_m = \mathcal{C}_{m,0} < \tilde{\mathcal{C}}_m,$$

where the right inequality follows by combining (E.1) with the fact that $\tilde{\mathcal{C}}_m \neq \mathcal{C}_{m,0}$. This demonstrates that $\mathcal{C}_{m,0}$ is closer to $\mathcal{C}_m$ than $\tilde{\mathcal{C}}_m$ is, which establishes the desired result.